What is R Programming Language?

R is a **programming language + software environment** mainly used for:

- Data Analysis
- Statistical Computing
- Data Visualization
- Machine Learning

It is widely used by:

- Data Scientists
- Statisticians
- Researchers

History of R Programming

Inventors

R was created by:

- Ross Ihaka
- Robert Gentleman

When was it created?

- Developed in **1993**
- First official release: **1995**

Where?

- At the **University of Auckland**, New Zealand

Why was R invented?

R was created to solve problems in **statistical computing**.

Main reasons:

- To provide a **free alternative** to expensive software like **S**
- To make **data analysis easier and flexible**
- To allow researchers to **write their own statistical methods**

In simple words:

R was built to make **data analysis powerful, free, and customizable**

Key Features of R

- Open-source (Free)
- Strong statistical capabilities
- Huge package ecosystem (CRAN)
- Excellent data visualization (graphs, charts)
- Works on Windows, Linux, Mac

How to Install R Language (Step-by-Step)

Step 1: Download R

Go to official website:

https://cran.r-project.org/

Steps:

1. Click **Download R**
2. Select your OS:
    - Windows / Mac / Linux
3. Click **Download R for Windows**
4. Click **base → Download R-x.x.x version**
5. Run the `.exe` file
6. Click **Next → Install**

Step 2: Verify Installation

After installing:

- Open **R Console**
- You should see something like:

```
R version 4.x.x (202x-xx-xx)
```

What is RStudio?

- RStudio is an **IDE (Integrated Development Environment)** for R
- It makes coding **easy and organized**

How to Install RStudio

Step 1: Download RStudio

https://posit.co/download/rstudio-desktop/

Steps:

1. Click **Download RStudio Desktop**
2. Choose **Free Version**
3. Download `.exe` file

Step 2: Install RStudio

1. Open downloaded file
2. Click **Next → Install → Finish**

Step 3: Open RStudio

- It will automatically detect R installation
- Interface has 4 parts:
    - Script Editor
    - Console
    - Environment
    - Files/Plots

First R Program

```
# Print message
print("Hello, R!")

# Simple calculation
a <- 10
b <- 5
sum <- a + b
print(sum)
```

Variables in R

A **variable** is a name used to store data in memory.

In R, we usually assign values using:

- <- (most common)
- = (also works)

Example:

```
x <- 10
name <- "Neeraj"
price = 99.5
```

Rules for Naming Variables in R

Follow these rules carefully:

Allowed Rules:

1. Must start with:
    - Letter (a–z, A–Z)
    - Or dot . (but not followed by number)
2. Can contain:
    - Letters

- Numbers
- Dot (.)
- Underscore (_)

Not Allowed:

1. Cannot start with number

```
2num <- 10   # Wrong
```

2. Cannot use spaces

```
my name <- "Neeraj"   # Wrong
```

3. Cannot use special characters

```
name@ <- "abc"   # Wrong
```

4. Cannot use reserved keywords
   Examples:

`if`, `else`, `repeat`, `function`, `TRUE`, `FALSE`

Valid Variable Names:

```
age <- 23
.myVar <- 10
total_marks <- 90
student1 <- "Ram"
```

Data Types in R

R supports multiple data types:

Numeric (Decimal Numbers)

Used for numbers with or without decimal

```
x <- 10
y <- 3.14
```

Integer

👉 Whole numbers with `L`

```
x <- 10L
```

Character (String)

Text values (must be in quotes)

```
name <- "Neeraj"
city <- 'Lucknow'
```

Logical (Boolean)

TRUE or FALSE values

```
isPassed <- TRUE
isFail <- FALSE
```

Complex

Numbers with imaginary part

```
z <- 3 + 2i
```

Check Data Type in R

Use `class()` function:

```
x <- 10
class(x)   # "numeric"

y <- "Hello"
class(y)   # "character"
```

Special Values in R

```
NA    # Missing value
NULL  # Empty value
Inf   # Infinity
NaN   # Not a number
```

Combined Example

```
# Variables
name <- "Neeraj"
age <- 23
marks <- 88.5
isStudent <- TRUE

# Print values
print(name)
print(age)

# Check type
class(age)

# Convert type
x <- "100"
num <- as.numeric(x)
print(num)
```

Type Casting in R (Type Conversion)

Type Casting means converting one data type into another.

Why Type Casting is Needed?

- To perform calculations
- To fix data type errors
- To prepare data for analysis

Built-in Type Casting Functions

Convert to Numeric

```
x <- "10"
as.numeric(x)   # 10
```

Convert to Character (String)

```
x <- 100
as.character(x)   # "100"
```

Convert to Logical

```
as.logical(1)   # TRUE
as.logical(0)   # FALSE
```

Convert to Integer

```
x <- 10.9
as.integer(x)   # 10 (decimal removed)
```

Convert to Complex

```
as.complex(5)   # 5+0i
```

Important Notes: Invalid Conversion

```
as.numeric("abc")   # NA with warning
```

Because "abc" cannot be converted into a number.

Decimal Loss in Integer

```
as.integer(5.9)   # 5 (not rounding)
```

Operators in R

Operators are symbols used to perform operations on variables and values.

Types of Operators in R

1. Arithmetic Operators

Used for mathematical operations:

```
a <- 10
b <- 3

a + b   # 13
a - b   # 7
a * b   # 30
a / b   # 3.33
a %% b  # 1 (remainder)
a %/% b # 3 (quotient)
a ^ b   # 1000 (power)
```

2. Relational Operators

Used for comparison (Result = TRUE/FALSE):

```
a <- 10
b <- 5

a > b    # TRUE
a < b    # FALSE
a == b   # FALSE
a != b   # TRUE
a >= b   # TRUE
a <= b   # FALSE
```

3. Logical Operators

Used for combining conditions:

```
x <- TRUE
y <- FALSE

x & y   # FALSE (AND)
x | y   # TRUE  (OR)
!x      # FALSE (NOT)
```

4. Assignment Operators

```
x <- 10
x = 10
10 -> x
```

5. Special Operators

i. Sequence Operator

```
1:5   # 1 2 3 4 5
```

ii. Membership Operator

```
x <- c(1,2,3)

2 %in% x   # TRUE
```

iii. Matrix Multiplication

```
# %*% is used for matrix multiplication
```

Collection Data Types

1. Vectors in R

A **vector** is the **basic data structure in R**
It stores **multiple values of the same data type**

Create a Vector

```
v <- c(1, 2, 3, 4, 5)
```

`c()` means **combine**

Types of Vectors

```
# Numeric vector
num <- c(10, 20, 30)

# Character vector
name <- c("Ram", "Shyam", "Mohan")

# Logical vector
log <- c(TRUE, FALSE, TRUE)
```

Access Elements

```
v <- c(10, 20, 30)

v[1]   # 10
v[2]   # 20
```

Vector Operations

```
a <- c(1,2,3)
b <- c(4,5,6)

a + b   # 5 7 9
a * b   # 4 10 18
```

Important Functions

```
v <- c(1, 2, 3, 4)

length(v)   # 4
sum(v)      # 10
mean(v)     # 2.5
```

Important Rule

All elements must be **same type**

```
x <- c(1, "A", TRUE)
# Converted to character automatically
```

2. Lists in R

A **list** can store **different data types together**

Create a List

```
my_list <- list(
  name = "Neeraj",
  age = 23,
  marks = c(80, 90, 85)
)
```

Access List Elements

```
my_list$name      # "Neeraj"
my_list[[2]]      # 23
```

Add Element

```
my_list$city <- "Lucknow"
```

List Example

```
l <- list(1, "Hello", TRUE, c(1,2,3))
```

List can store:

- Numbers
- Text
- Logical
- Even vectors inside it

3. Data Frames in R

A **data frame** is like a **table (Excel sheet)**

- Rows = records
- Columns = variables

Create Data Frame

```
data <- data.frame(
  name = c("Ram", "Shyam", "Mohan"),
  age = c(20, 21, 22),
  marks = c(80, 85, 90)
)
```

View Data

```
print(data)
```

Access Data

```
data$name       # column
data[1, ]       # first row
data[ ,2]       # second column
data[1,2]       # specific value
```

Add Column

```
data$city <- c("Delhi", "Lucknow", "Kanpur")
```

Structure of Data Frame

```
str(data)
```

Summary

```
summary(data)
```

Difference Between Vector, List, Data Frame

Feature	Vector	List	Data Frame
Data Type	Same	Different	Column-wise same
Structure	1D	Complex	2D Table
Example	c(1,2,3)	list(1,"A")	table

Conditional Statements in R

Conditional statements are used to **make decisions in a program** based on conditions.

1. `if` Statement

Executes code **only if condition is TRUE**

```
x <- 10

if (x > 5) {
  print("x is greater than 5")
}
```

2. if...else Statement

Executes one block if TRUE, otherwise another block

```
x <- 3

if (x > 5) {
  print("Greater")
} else {
  print("Smaller")
}
```

3. if...else if...else

Used when you have **multiple conditions**

```
marks <- 75

if (marks >= 90) {
  print("Grade A")
} else if (marks >= 60) {
  print("Grade B")
} else {
  print("Fail")
}
```

4. Nested if

One `if` inside another `if`

```
x <- 10

if (x > 5) {
  if (x < 15) {
    print("x is between 5 and 15")
  }
}
```

5. ifelse () Function (Important)

Used for **vectorized condition checking** (very powerful in R)

```
x <- c(1, 2, 3, 4, 5)

result <- ifelse(x > 3, "Big", "Small")
print(result)
```

Output:

```
"Small" "Small" "Small" "Big" "Big"
```

6. switch Statement

Used when you have **multiple fixed options**

```
choice <- 2

result <- switch(choice,
  "One",
```

```
  "Two",
  "Three"
)

print(result)   # "Two"
```

Important Rules

- Condition must return **TRUE or FALSE**
- Use `{ }` for multiple statements
- Indentation improves readability

What are Loops in R?

Loops are used to **repeat a block of code multiple times**.

Types of Loops in R

1. `for` loop
2. `while` loop
3. `repeat` loop

1. `for` Loop

Used when the **number of iterations is known**

Example 1: Print Numbers

```
for (i in 1:5) {
  print(i)
}
```

Output: 1 2 3 4 5

Example 2: Sum of Numbers

```
sum <- 0

for (i in 1:5) {
  sum <- sum + i
}

print(sum)   # 15
```

2. `while` Loop

Used when condition is checked **before execution**

Example

```
i <- 1
```

```
while (i <= 5) {
  print(i)
  i <- i + 1
}
```

3. repeat Loop

Runs **infinite loop** until you stop it using `break`

Example

```
i <- 1

repeat {
  print(i)
  i <- i + 1

  if (i > 5) {
    break
  }
}
```

Loop Control Statements

1. break → Stop loop immediately

```
for (i in 1:10) {
  if (i == 5) {
    break
  }
  print(i)
}
```

Output: 1 2 3 4

2. next → Skip current iteration

```
for (i in 1:5) {
  if (i == 3) {
    next
  }
  print(i)
}
```

Output: 1 2 4 5

Difference Between Loops

Loop	Condition Check	Use Case
for	Fixed times	Known iterations
while	Before loop	Condition-based
repeat	Inside loop	Infinite until break